Luna's Pharmacy

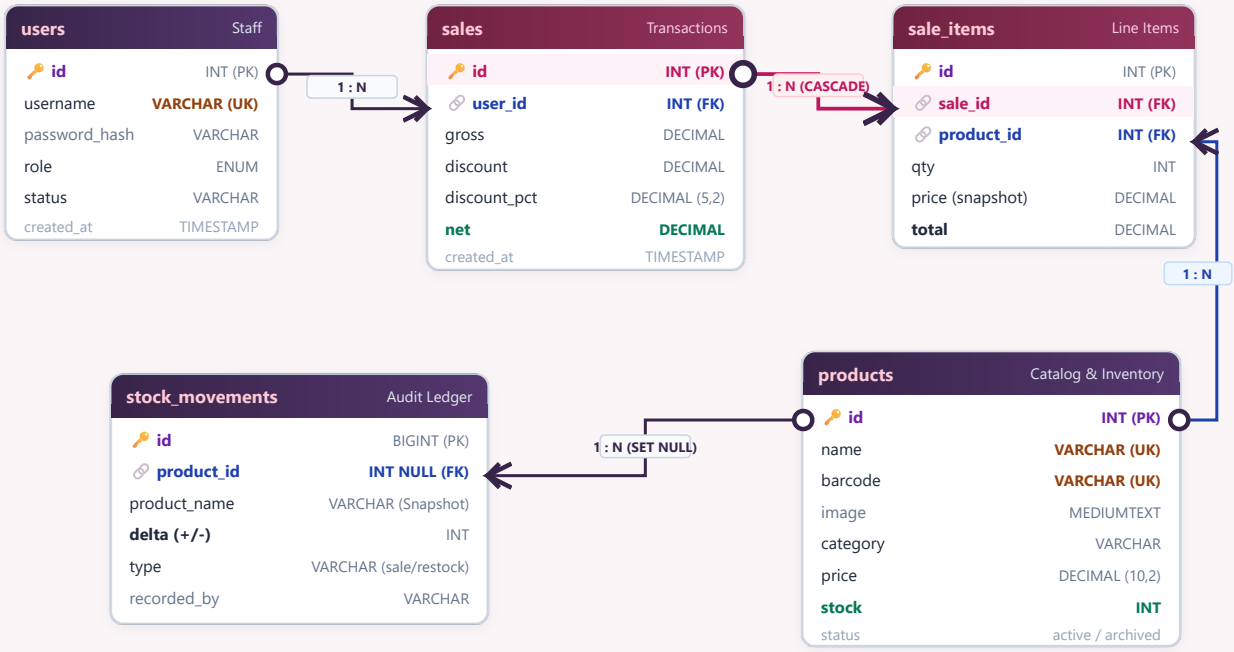
Database Architecture, Entity-Relationship Diagram (ERD) & Normalization Report

Database Engine: MySQL / TiDB Cloud Design Level: 3rd Normal Form (3NF) Application: POS & Inventory Management

1. ENTITY-RELATIONSHIP DIAGRAM (ERD)

VISUAL LAYOUT

The diagram below displays the 5 core relational entities, their primary keys (PK), foreign keys (FK), attributes, and exact cardinalities.



2. RELATIONAL KEYS & CARDINALITY MATRIX

REFERENTIAL INTEGRITY

PARENT ENTITY (PK)	CHILD ENTITY (FK)	CARDINALITY	DELETE BEHAVIOR	BUSINESS PURPOSE
`users.id` PK	`sales.user_id` FK	1 : Many (1:N)	RESTRICT	Tracks which cashier or admin authenticated and completed the transaction.
`sales.id` PK	`sale_items.sale_id` FK	1 : Many (1:N)	CASCADE	Itemized breakdown of medicines bought in a single sale receipt.
`products.id` PK	`sale_items.product_id` FK	1 : Many (1:N)	RESTRICT	Links line item to product master catalog. Prevents deleting active products with sales history.

PARENT ENTITY (PK)	CHILD ENTITY (FK)	CARDINALITY	DELETE BEHAVIOR	BUSINESS PURPOSE
`products.id` PK	`stock_movements.product_id` FK	1 : Many (1:N)	`SET NULL`	Audit log of all inventory shifts (+restock, -sale, manual edits). Preserves name if deleted.

3. COMPREHENSIVE DATABASE NORMALIZATION REPORT

THEORY & VALIDATION

The Luna's Pharmacy database schema has been designed following strict relational modeling principles to ensure data integrity, prevent anomalies, and optimize transaction performance.

First Normal Form (1NF) — Elimination of Repeating Groups

✓ Compliant

Criteria: Each column contains atomic (single, indivisible) values, and each record is uniquely identifiable by a primary key.

- **Atomic Data:** All columns store single primitive scalar values (integers, fixed-point decimals, strings, timestamps).
- **No Multi-Valued Fields:** Multiple purchased medicines are never stored as comma-delimited strings or JSON lists within `sales`. Instead, each individual purchased product is placed in a dedicated row in the child table `sale_items`.
- **Unique Rows:** Every table incorporates an explicit auto-incrementing Primary Key (`id`).

Second Normal Form (2NF) — Elimination of Partial Dependencies

✓ Compliant

Criteria: The table must be in 1NF, and all non-key attributes must depend on the *entire* primary key (no partial functional dependency).

- **Single-Column Primary Keys:** All 5 tables utilize a single-column surrogate primary key (`id INT/BIGINT AUTO_INCREMENT PRIMARY KEY`).
- **Zero Partial Dependencies:** Because none of the tables use a composite (multi-column) primary key, partial key dependencies are mathematically impossible. Every non-key attribute is fully dependent upon the single primary key `id`.

Third Normal Form (3NF) — Elimination of Transitive Dependencies

✓ Compliant

Criteria: The table must be in 2NF, and no non-key attribute can depend on another non-key attribute ($X \to Y \to Z$). Non-key columns must depend *only* on the primary key.

- **Independent Entities:** Staff profile attributes (role, status) reside strictly in `users`; product master data (barcode, price, stock) reside strictly in `products`.
- **Referential Links Only:** Relational tables store only foreign keys (`user_id`, `sale_id`, `product_id`) rather than repeating vendor or staff details.

4. SPECIALIZED RETAIL & POS DESIGN PATTERNS

INDUSTRY BEST PRACTICE

In retail pharmacy and point-of-sale environments, certain apparent redundancies are deliberate, standard architectural decisions required for legal, fiscal, and audit immutability:

1. Point-in-Time Historical Price Freeze (`sale_items.price`)

While `products` has a `price` field, `sale_items.price` captures the *exact selling price at the moment of checkout*. If the pharmacy adjusts the retail price of Paracetamol next month from ₱8.00 to ₱10.00, past receipts and monthly tax reports must legally remain at ₱8.00. Therefore, `sale_items.price` is functionally dependent on the transaction line item, not the current master catalog price.

2. Financial Ledger Immutability & Derived Totals (`total, gross, net`)

In pure academic relational theory, derived attributes ($total = qty \times price$, $net = gross - discount$) can be

computed dynamically. However, in production financial systems, pre-computing and persisting totals guarantees rounding precision, prevents historical discrepancies caused by floating-point math, and delivers instant aggregation performance across millions of sales records without scanning every child item.

3. Resilient Audit Trail Snapshots (`stock_movements.product_name`)
The `stock_movements` table uses `ON DELETE SET NULL` for `product_id`, while simultaneously retaining `product_name`. If a discontinued drug is removed from the catalog, the pharmacy's inventory audit trail preserves complete historical clarity regarding what stock arrived and departed.

5. SUMMARY FOR ACADEMIC & TECHNICAL DEFENSE

QUICK REFERENCE

QUESTION / CRITERION	TECHNICAL ANSWER
What Normal Form is the database in?	The database is in **Third Normal Form (3NF)**.
Why is `price` in both `products` and `sale_items`?	It is a *Point-in-Time Historical Snapshot* to protect fiscal records from future price modifications.
Why store `net` and `gross` in `sales`?	For transaction ledger immutability and high-speed reporting performance.
How is referential integrity enforced?	Via foreign keys with `CASCADE` (sales → items), `RESTRICT` (products → items), and `SET NULL` (products → stock audit).